Two-Wave Extreme Rejection — Algorithmic Specification v1.0

Model & Arm strategy · deterministic translation for MetaTrader 5 · pre-coding document for approval
Source of truth: "Two-Wave Extreme Rejection — Model & Arm Strategy" manual (EN + AR). Descriptive working name, not the trader's official name.

Scope of this document. No trading code is delivered here. This is the specification that must be approved before the detector is written. It converts each discretionary concept into a computable definition and states, for every number, whether it comes from the trader or from us. Nothing in it claims the strategy is profitable: the 33-trade historical sample (17 W / 13 L / 3 BE) is a sample, not a forecast.

Evidence classes used throughout

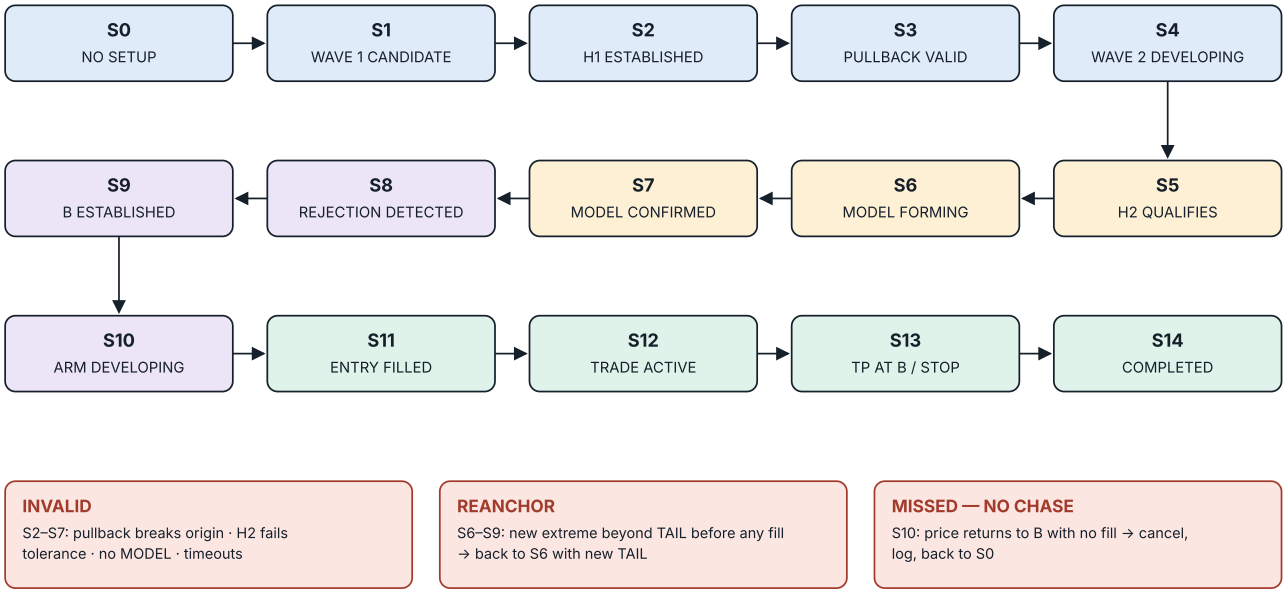
PROVEN / OBSERVED · STRONG INFERENCE · MEASURED · BEHAVIORAL INFERENCE · HYPOTHESIS · TEST VALUE · UNRESOLVED

Reading rule. Every threshold marked TEST VALUE is a research parameter exposed as an EA input. It is *not* a trader rule. Changing one creates a new, separately logged test population; results from different parameter sets are never pooled.

1. Conventions and causal basis

- **Closed bars only for structure.** All structural decisions (swings, MODEL, rejection, B, ARM) are evaluated once per new bar on bars with index ≥ 1. The forming bar (index 0) is never used for structure.
- **Ticks only for execution.** Fills, stop and target are evaluated on ticks, using the correct side of the quote (§16), never on bar closes.
- **Normaliser.** `ATR = iATR(symbol, tf, AtrPeriod)` read at bar 1. TEST VALUE AtrPeriod = 14. ATR is our scale-free ruler across instruments; the trader never mentions it.
- **SELL is described; BUY is the exact mirror** (swap high/low, max/min, above/below).
- **R** = |TAIL − B|. All depth percentages are measured from B (0 %) to TAIL (100 %).

2. State machine



Every transition writes: setup ID · from-state · to-state · server time · bar time · bid/ask · reason code. BUY uses the exact mirror (L1, L2, tail = lowest wick).

State	Entered when (SELL)	Leaves to	Alert	Drawn at entry
S0 No setup	Start, or previous setup closed	S1	—	—
S1 Wave 1 candidate	Confirmed swing low O exists and price has risen ≥ MinWave1ATR × ATR above it	S2 · S0	1	Wave 1 line (provisional, dashed)
S2 H1 established	Swing high H1 confirmed by the causal swing rule (§3)	S3 · INVALID	—	Wave 1 line solid, H1 label + price
S3 Pullback valid	Swing low P confirmed with P > O and depth ≥ MinPullback	S4 · INVALID	—	Pullback line, "PULLBACK — VALID"
S4 Wave 2 developing	Close above P + RevMult × ATR after P	S5 · INVALID	2	Wave 2 line (grows per bar), H1 reference line
S5 H2 qualifies	Running high X ≥ H1 − SecondExtremeTolerance × Wave1Range	S6	3	H1↔H2 bracket, "SECOND EXTREME: VALID"
S6 Model forming	≥ 1 closed bar inside the model band at X (§5)	S7 · REANCHOR	—	Model rectangle (dashed)
S7 Model confirmed	Cluster ≥ MinModelCandles and a bar closes below BODY_LOW (cluster frozen)	S8 · REANCHOR	4	Model rectangle solid, BODY band, TAIL line
S8 Rejection detected	Counter-move from BODY_LOW ≥ RejClearATR × ATR	S9 · REANCHOR	5	Rejection arrow
S9 B established	Running low L* confirmed as swing low (§7)	S10 · S8 · REANCHOR	6, 9	B line, entry lines, entry zone, stop, R:R
S10 ARM developing	Close ≥ B + ArmStartFrac × (E1 − B)	S11 · MISSED	7, 8	ARM arrow
S11 Entry filled	Tick reaches E1 (virtual in V1, real order in V3)	S12	10	Fill marker
S12 Trade active	Position (virtual or real) open	S13	—	—
S13 TP / Stop	Tick reaches B or SL	S14	11 / 12	Exit marker, result label
S14 Completed	Row written to research log, screenshot saved	S0	—	Objects kept, greyed

3. Wave 1, pullback, Wave 2

The manual establishes that two waves separated by a pullback exist PROVEN / OBSERVED and that the pullback must not break the Wave 1 origin PROVEN / OBSERVED. It gives **no numerical definition** of a wave UNRESOLVED. Four deterministic candidates:

Version	Definition	Causal?	Pros	Cons
A · Reversal swing	A high is a confirmed swing high once a later bar *closes* ≥ RevMult × ATR below it (mirror for lows).	Yes — confirmed at the confirming bar's close	Scale-free, one parameter, adapts to volatility	Confirmation lags by the reversal size
B · Minimum movement	Wave = move of ≥ MinWave1ATR × ATR between two Version-A swings.	Yes	Filters noise legs	Second parameter
C · N-candle swing	Highest high of the last N bars.	Yes	Simple	N has no basis; instrument-dependent
D · Fractal	Bar whose high exceeds K bars on each side.	Only after K bars	Familiar	Fixed lag of K bars, ignores size

Recommended first research implementation: A + B. Swings by ATR-reversal, wave accepted only above a minimum ATR size. TEST VALUE RevMult = 1.5 · MinWave1ATR = 3.0 · MinPullbackFrac = 0.20 of Wave 1 range · MaxPullbackFrac = 0.95. TEST PARAMETERS — NOT TRADER-STATED. Versions C and D stay selectable by the `SwingMode` input for comparison.

```
O   = last confirmed swing low before the rise                      # Wave 1 origin
H1  = first confirmed swing high after O with (H1 - O) >= MinWave1ATR * ATR
P   = first confirmed swing low after H1
valid_pullback  = (P > O) and MinPullbackFrac <= (H1 - P)/(H1 - O) <= MaxPullbackFrac
if low of any bar after H1 <= O:   SETUP = INVALID, reason = PULLBACK_BROKE_ORIGIN
Wave 2 = rise from P; X = running max(high) since P; iX = bar of X
```

Second extreme

```
Wave1Range = H1 - O
gap        = H1 - X                      # negative when H2 exceeds H1
gap_pct_w1 = gap / Wave1Range            # logged
gap_pct_w2 = gap / (X - P)               # logged as alternative denominator
qualifies  = gap_pct_w1 <= SecondExtremeTolerance
```

TEST VALUE SecondExtremeTolerance = 0.10 (the closest accepted example was ≈10 % short MEASURED; which range that 10 % refers to is itself UNRESOLVED, so both denominators are logged). No upper limit on how far H2 may exceed H1 is established UNRESOLVED; `MaxOvershootFrac` exists and defaults to off. If Wave 2 turns down (a swing high is confirmed) while `qualifies` is false → INVALID, reason `H2_TOO_LOW`, with H1, H2, gap and both percentages in the log.

4. MODEL detection in real time

The MODEL is the candle cluster at the end of Wave 2; the trader labels it himself PROVEN / OBSERVED. Its body zone carries the entries and its tail carries the stop STRONG INFERENCE. The number of candles and the cluster boundary are not defined by the trader UNRESOLVED.

```
band_low   = X - ModelBandATR * ATR                  # X = current extreme of Wave 2 (tail candidate)
in_band(b) = max(open_b, close_b) >= band_low        # bar body reaches the band
cluster    = maximal run of consecutive closed bars that contains iX and all satisfy in_band

MODEL_START, MODEL_END = first / last bar of cluster
MODEL_TAIL      = max(high)  over cluster             # == X
MODEL_HIGH      = MODEL_TAIL
MODEL_LOW       = min(low)   over cluster
MODEL_BODY_HIGH = max(max(open,close)) over cluster   # deeper body edge (SELL)
MODEL_BODY_LOW  = min(min(open,close)) over cluster   # near body edge   (SELL)

S6 FORMING   : cluster has >= 1 bar and S5 holds
S7 CONFIRMED : MinModelCandles <= len(cluster) <= MaxModelCandles
               and the newest closed bar closes below MODEL_BODY_LOW      -> cluster is frozen
REANCHOR     : any high > MODEL_TAIL before a fill -> discard levels, X = new high, back to S6
NO MODEL     : price leaves with len(cluster) < MinModelCandles -> INVALID, reason NO_MODEL
```

TEST VALUE ModelBandATR = 1.0 · MinModelCandles = 3 · MaxModelCandles = 8. These are starting values for research, chosen so the documented clusters fit; they are not observed constants. The model is only *confirmed* when price closes out of it, so the rectangle becomes final on that bar and never changes afterwards except through REANCHOR, which is logged as a new model revision.

5. Rejection

Seven readable trades were entered after a clear rejection; three were sold at the top after a tiny dip; two arrival-style ladders exist MEASURED. Waiting for rejection is the dominant behaviour STRONG INFERENCE; whether it is mandatory is UNRESOLVED.

```
Lstar  = running min(low) since MODEL_END
move   = MODEL_BODY_LOW - Lstar
REJECTION = YES        if move >= RejClearATR * ATR
          = NO         if move <  RejTinyATR  * ATR
          = AMBIGUOUS  otherwise
```

Mode	Behaviour	Population
A · STRICT REJECTION (default)	Setup proceeds to S9 only with REJECTION = YES. AMBIGUOUS and NO are logged as NO TRADE, reason `NO_STRICT_REJECTION`.	`pop=STRICT`

Mode	Behaviour	Population
B · ARRIVAL (log-only)	Records setups where price sits at the extreme without a clear rejection, with the levels it *would* have used. Never produces an order in any version until an arrival rule is specified and approved.	`pop=ARRIVAL`

TEST VALUE RejClearATR = 2.0 · RejTinyATR = 0.75. The two populations are written to separate files and never mixed in any statistic.

6. B and ARM without future information

B is the end of the first clear counter-move away from the MODEL and the primary target PROVEN / OBSERVED. It is not always the low between the two waves MEASURED.

```
# S8 -> S9 : B is only known after price has turned
B_candidate = Lstar
B confirmed when a bar closes >= Lstar + max(BConfirmATR * ATR, BConfirmFrac * (MODEL_BODY_LOW - Lstar))
B = Lstar ; B_time = bar of Lstar ; B_confirm_time = confirming bar

# while in S9 (no ARM yet): a new low below B  -> B is not final, return to S8 and keep tracking
# S9 -> S10 : ARM
ARM starts when a bar closes >= B + ArmStartFrac * (E1 - B)
ARM_bars   = bars since B_time ; ARM_max_bar_share = largest single-bar range / (current - B)

# in S10 with no fill:
if bid <= B:                       SETUP = MISSED, alert "SETUP MISSED - NO CHASE", cancel, S0
if bars since B_confirm_time > MaxBarsBToFill:   SETUP = EXPIRED
```

TEST VALUE BConfirmATR = 1.0 · BConfirmFrac = 0.25 · ArmStartFrac = 0.50 · MaxBarsBToFill = 60. B is published on the confirming bar, never on the bar of the low itself, so the B line appears on the chart with a visible lag: that lag is the price of having no look-ahead. Once any entry is filled, B is frozen for that setup.

7. Entry, stop, target

Item	Definition (SELL)	Class
Entry method A · body edge (default)	E1 = MODEL_BODY_LOW (near edge) · E2 = MODEL_BODY_HIGH (deeper edge)	MEASURED body-edge match in 5 readable cases; rule itself HYPOTHESIS
Entry method B · percentage	E1 = B + EntryPct1 × R · E2 = B + EntryPct2 × R	TEST VALUE 0.68 / 0.79 = midpoints of the measured 61–75 % and 73–85 % ranges
Entry 3	Disabled by default. If enabled for research: E3 = MODEL_TAIL (beyond the prior extreme in 2 of 3 observed cases).	UNRESOLVED
Number of entries	`EntryCount` ∈ {1, 2, 3}. Observed 1–3, never more than 3, no mandatory number.	PROVEN / OBSERVED range · count rule UNRESOLVED
Order type	Pending limit placed when B is confirmed (S9), before price arrives.	PROVEN / OBSERVED
Stop	SL = MODEL_TAIL + buffer. Location beyond the tail is established; the distance is not.	STRONG INFERENCE location · buffer UNRESOLVED
Target	TP = B for every entry of the setup. No fixed R multiple.	PROVEN / OBSERVED
Extended target	`UseExtendedTarget = false`.	UNRESOLVED
Breakeven / trailing / partials	Not implemented. Three break-even results exist in the ledger but no rule.	UNRESOLVED

Stop buffer modes

StopBufferMode	SL (SELL)	Default
EXACT_TAIL	TAIL + spread + 1 point	

StopBufferMode	SL (SELL)	Default
R_BASED	TAIL + StopBufferR × R	TEST VALUE 0.2 — single reading of 0.19R
ATR_BASED	TAIL + StopBufferATR × ATR	0.5
FIXED_POINTS	TAIL + StopBufferPoints	per symbol
PERCENT	TAIL × (1 + StopBufferPct)	off

Arithmetic that must be seen before choosing a default. With entry at depth d and buffer k·R, R:R = d / (1 − d + k). At the measured Entry 1 depths (61–75 %): **k = 0.2 gives R:R 1.03–1.67**; k = 0 gives 1.56–3.00. At Entry 2 depths (73–85 %): k = 0.2 gives 1.55–2.43; k = 0 gives 2.70–5.67. The R:R measured on the trader's own trades was 1.9–3.6. That range is reproduced by a stop close to the tail, not by a 0.2R buffer on Entry 1. So the 0.2R reading and the measured R:R are in tension, and the series also shows tolerated tail breaches in other periods. **V1 therefore logs EXACT_TAIL and R_BASED(0.2) side by side for every setup** and neither is called the trader's rule.

```
geometry is valid only if:
  E1 > B + MinRewardSpreads * spread                    # target not inside the spread
  SL - E_deepest >= SYMBOL_TRADE_STOPS_LEVEL * point
  E_k distinct by >= 2 * spread (else collapse to one entry)
RR_k = (E_k - B) / (SL - E_k)                           # computed after E, SL, B exist; logged, never imposed
```

8. No-trade conditions and reason codes

Code	Condition	Detected in
H2_TOO_LOW	Second extreme short of H1 by more than SecondExtremeTolerance when Wave 2 turns	S4
PULLBACK_BROKE_ORIGIN	Any low ≤ O between H1 and H2	S2–S5
PULLBACK_OUT_OF_RANGE	Pullback depth outside [MinPullbackFrac, MaxPullbackFrac]	S3
NO_MODEL	Price leaves the extreme with fewer than MinModelCandles in the band	S6
MODEL_UNRELIABLE	Cluster longer than MaxModelCandles, or body zone thinner than 2 × spread	S6–S7
NO_STRICT_REJECTION	REJECTION ≠ YES in Mode A	S8
NO_B	No confirmed B within MaxBarsModelToB	S8
ENTRY_AMBIGUOUS	E1 ≤ B, or entries collapse and reward < MinRewardSpreads × spread	S9
STOP_INVALID	Stop distance below broker stop level, or volume cannot be sized	S9
MISSED_NO_CHASE	Price returns to B before any fill	S10
EXPIRED	MaxBarsBToFill exceeded	S10
SPREAD / SESSION / SAFETY	Spread above limit, market closed, or a safety limit active (§12)	any

Every rejected setup is kept on the chart, marked INVALID / NO TRADE / MISSED with its reason, and written to the log with the same detail as an accepted one.

9. Research classification and Rule Match Score

A++ / A+ / B are our research labels (TEST VALUE), never named by the trader. The dashboard prints “RESEARCH CLASSIFICATION” beside them and shows a **Rule Match Score**, not a probability.

Component	Full points when	Points
Second extreme	H2 > H1 (exceeds) = 20 · within tolerance = 10	20
Model	MinModelCandles ≤ n ≤ MaxModelCandles and body zone ≥ 3 × spread	15
Rejection	YES = 20 · AMBIGUOUS = 8	20
B	Confirmed on first attempt (no B revision)	15

Component	Full points when	Points
ARM	ARM_bars ≥ ArmGradualBars and no bar > 50 % of the ARM	15
Entries	Two distinct body edges readable	15

TEST VALUE A++ ≥ 90 · A+ 70–89 · B 50–69 · below 50 or any §8 code = NO TRADE · ArmGradualBars = 4. It is not established that A++ outperforms A+; the score exists so that this can be tested.

10. Chart drawing architecture — the visual reconstruction engine

Drawing is a core requirement, not decoration. Each object is created by the state transition that makes its information available, and by nothing else. Object names are `TWER_<setupId>_<part>` so a setup can be redrawn, greyed or removed as a unit and survives a restart.

#	Element	MT5 object	Appears at	Label / content
1	Wave 1	OBJ_TREND O→H1 + OBJ_TEXT	S1 dashed, S2 solid	WAVE 1 · H1 = price, time
2	Pullback	OBJ_TREND H1→P (grey)	S3	PULLBACK — VALID, or PULLBACK — INVALID · Wave 1 origin broken
3	Wave 2	OBJ_TREND P→X, end point updated per bar	S4	WAVE 2 · H2 = price
4	Second extreme	OBJ_TREND horizontal at H1 from H1 to H2 bar + bracket	S4 / S5	H1 · H2 · SECOND EXTREME: VALID / INVALID (gap %)
5	MODEL zone	OBJ_RECTANGLE MODEL_START..END × MODEL_LOW..TAIL	S6 dashed, S7 solid	MODEL ZONE
6	MODEL body	OBJ_RECTANGLE (filled) BODY_LOW..BODY_HIGH + two edge lines	S7	MODEL BODY · Near body edge · Deeper body edge
7	MODEL tail	OBJ_TREND horizontal from tail bar, ray right	S7	MODEL TAIL = price
8	Rejection	OBJ_ARROWED_LINE model → Lstar	S8	REJECTION CONFIRMED (YES / AMBIGUOUS)
9	B	OBJ_TREND horizontal, thick, from B bar, ray right	S9 (confirming bar)	B · TARGET · TP = B price
10	ARM	OBJ_ARROWED_LINE B → current, different colour and style from #8	S10	ARM
11	Entry lines	OBJ_TREND horizontal per entry	S9	SELL LIMIT 1 price · SELL LIMIT 2 price
12	Entry zone	OBJ_RECTANGLE transparent, body zone extended right to current bar	S9	ENTRY ZONE
13	Stop	OBJ_TREND horizontal + short vertical connector from the tail line	S9	STOP LOSS price · “0.2R TEST VALUE” when R_BASED
14	Target	Same line as B, restyled	S9	TP = B price
15	Risk / reward	Two OBJ_RECTANGLE to the right of price: entry→stop, entry→B	S9	R:R = X.XX (computed from entry, stop, B)
16	Fill / exit	OBJ_ARROW at tick price and time	S11 / S13	FILLED · TP HIT · STOP HIT
17	Verdict	OBJ_TEXT above model	INVALID / MISSED	SETUP INVALID · Reason: … / MISSED ENTRY — NO CHASE
18	Dashboard	OBJ_LABEL block, chart corner	always	Strategy, symbol, TF, state, direction, H1, H2, MODEL, REJECTION, B, ENTRY, STOP, TP, R:R, class, score, setup ID

- **No future information.** B is drawn on its confirming bar, the ARM arrow when ARM starts, entry lines when B exists. Nothing is back-dated except the anchor times of objects whose price was fixed earlier (for example the B line starts at the bar of the low but is created on the confirming bar; the log stores both times).
- **Alert ↔ chart parity.** One function, `Transition(setup, newState, reason)`, performs draw → log → alert in that order. A notification cannot be sent for a state whose objects were not created.
- **Lifetime.** Entry lines stay until filled, expired, missed or invalidated. Finished, invalid and missed setups are kept and greyed; `KeepHistorySetups` (default 20) limits clutter.
- **Visual language.** Six colours: waves blue, pullback grey, model/entries amber, rejection red arrow, ARM violet arrow, B/TP green, stop red line. No other decoration.

- **Snapshots.** `ChartScreenShot()` saves a PNG named by setup ID at MODEL, REJECTION, B, ARM and FULL SETUP into `MQL5/Files/TWER/shots/`. MT5 push notifications are text-only, so the push carries the setup ID (e.g. `#EURUSD-M5-20261007-001`); sending the image itself needs the optional Telegram channel (§11).

11. Alert architecture

#	Alert	Fired by	Default channel
1	Potential Wave 1 detected	→S1	terminal log
2	Wave 2 developing	→S4	terminal log
3	Potential Two-Wave Extreme setup detected	→S5	push
4	MODEL FORMED	→S7	push + screenshot
5	REJECTION CONFIRMED	→S8	push + screenshot
6	B IDENTIFIED	→S9	push + screenshot
7	ARM DEVELOPING	→S10	push + screenshot
8	ENTRY ZONE APPROACHING	price within ApproachATR × ATR of E1	push
9	LIMIT ORDER PLACED (virtual in V1)	→S9 after geometry valid = FULL SETUP	push + screenshot
10	ENTRY FILLED	→S11	push
11	STOP HIT	→S13	push
12	TP AT B HIT	→S13	push
13	SETUP MISSED — NO CHASE	MISSED	push + screenshot

- **Channels.** `Alert()` popup, `SendNotification()` to the MT5 mobile app (needs the MetaQuotes ID set in the terminal), optional Telegram bot through `WebRequest` (the URL must be whitelisted in terminal options) for full text and screenshots.
- **Platform limits.** A push message is limited to 255 characters and MetaQuotes throttles frequency (no more than about 2 per second and 10 per minute). So the push is one compact line — `#ID SELL EURUSD M5 | S9 B IDENTIFIED | E 1.17269 SL 1.17392 TP 1.17110 | RR 1.29 | A+ 78` — and the full field list (H1, H2, model, tail, B, ARM, quality, reason) goes to the log, the dashboard and Telegram. A per-minute token bucket queues excess messages; per-stage on/off inputs keep early-stage noise off the phone.
- **No repeats.** Alerts are emitted only by state transitions; each (setup ID, alert number) pair is persisted and can fire once.

12. Data structures

```
enum ENUM_TWER_STATE { S0_NONE, S1_W1_CANDIDATE, S2_H1, S3_PULLBACK, S4_W2, S5_H2_OK, S6_MODEL_FORMING,
S7_MODEL_OK,
                       S8_REJECTION, S9_B, S10_ARM, S11_FILLED, S12_ACTIVE, S13_EXIT, S14_DONE,
                       SX_INVALID, SX_MISSED, SX_EXPIRED };
enum ENUM_TWER_REJ   { REJ_NO, REJ_AMBIGUOUS, REJ_YES };
enum ENUM_TWER_MODE  { MODE_ALERT_ONLY, MODE_DEMO, MODE_LIVE };

struct TwerSwing  { datetime time; double price; bool isHigh; datetime confirmTime; };
struct TwerModel  { datetime tStart, tEnd, tTail; double tail, high, low, bodyHigh, bodyLow; int candles; int
revision; };
struct TwerEntry  { double price; double volume; ulong ticket; bool filled; datetime fillTime; double fillPrice;
double rr; };
struct TwerSetup  {
   string   id;                 // SYMBOL-TF-YYYYMMDD-NNN
   string   symbol; ENUM_TIMEFRAMES tf; int dir;         // -1 SELL, +1 BUY
   string   population;         // STRICT | ARRIVAL
   ENUM_TWER_STATE state; datetime stateTime;
   TwerSwing origin, ext1, pullback, ext2;               // O, H1/L1, P, H2/L2
   double   gapPctW1, gapPctW2;
   TwerModel model;
   ENUM_TWER_REJ rejection; double rejMoveATR;
   double   B; datetime tB, tBConfirm; int bRevisions;
   datetime tArmStart; int armBars; double armMaxBarShare;
   TwerEntry e[3]; int entryCount;
   double   slExactTail, slTest, tp, R, atr, spreadAtSignal;
   int      score; string grade; string reason;          // reason code on accept or reject
   double   mae, mfe; datetime tExit; string result;     // TP | SL | MISSED | INVALID | EXPIRED
   uint     alertsSentMask;
};
struct TwerSymbolCfg { string logical, broker; int digits; double point, tickSize, tickValue, contractSize,
volMin, volStep, volMax;
                       int stopsLevel; double maxSpreadPts; string sessions; bool enabled; };
```

13. Configurable parameters

Input	Default	Class
`SwingMode`	A_REVERSAL	TEST VALUE
`AtrPeriod`	14	TEST VALUE
`RevMult`	1.5	TEST VALUE
`MinWave1ATR`	3.0	TEST VALUE
`MinPullbackFrac / MaxPullbackFrac`	0.20 / 0.95	TEST VALUE
`SecondExtremeTolerance`	0.10	TEST VALUE
`MaxOvershootFrac`	off	UNRESOLVED
`ModelBandATR`	1.0	TEST VALUE
`MinModelCandles / MaxModelCandles`	3 / 8	TEST VALUE
`RejectionMode`	STRICT	STRONG INFERENCE
`RejClearATR / RejTinyATR`	2.0 / 0.75	TEST VALUE
`BConfirmATR / BConfirmFrac`	1.0 / 0.25	TEST VALUE
`ArmStartFrac`	0.50	TEST VALUE
`ArmGradualBars`	4	TEST VALUE

Input	Default	Class
`EntryPct1 / EntryPct2`	0.68 / 0.79	TEST VALUE
`EntryCount`	1	UNRESOLVED
`StopBufferMode`	log EXACT_TAIL and R_BASED	UNRESOLVED
`StopBufferR`	0.2	TEST VALUE
`UseExtendedTarget`	false	UNRESOLVED
`MinRewardSpreads`	3	TEST VALUE
`Timeframes`	M5 (M1, M15, H1 as separate populations)	PROVEN / OBSERVED
`OperatingMode`	ALERT_ONLY	TEST VALUE
`RiskPercentPerSetup`	0.25	TEST VALUE
`MaxOpenSetups / PerSymbol`	2 / 1	TEST VALUE
`MaxDailyLossPct`	1.0	TEST VALUE
`MaxConsecutiveLosses`	3	TEST VALUE

Input	Default	Class
`ApproachATR`	0.5	TEST VALUE
`MaxBarsModelToB / MaxBarsBToFill`	60 / 60	TEST VALUE
`EntryMethod`	BODY_EDGE	HYPOTHESIS

Input	Default	Class
`MaxSpreadPoints`	per symbol	TEST VALUE
`MaxSlippagePoints`	per symbol	TEST VALUE
`Alert switches 1–13, UseTelegram, Screenshots`	see §11	TEST VALUE
`KeepHistorySetups`	20	TEST VALUE

Risk and safety inputs are safety controls of this project. They say nothing about the original trader, whose observed 2–5 % losses per trade are deliberately not copied.

14. Unresolved rules and how each is handled

#	Unresolved rule	Handling in v1	Evidence that would settle it
1	Stop buffer and stop discipline	Both EXACT_TAIL and 0.2R logged per setup; hard stop only	Stop prices on 10+ more trader trades
2	Exact entry price rule	Body-edge default; percentage method logged in parallel	Order prices vs candle bodies on unseen trades
3	Is rejection mandatory?	STRICT traded later; ARRIVAL log-only, separate population	Blind test on unseen trades
4	Wave / pullback minimum size	ATR test values; every rejected candidate logged with sizes	Human-vs-bot mismatches
5	Model boundary and candle count	Band + min/max candles as test values; model revisions logged	Trader-drawn model boxes
6	Number of entries and volume split	EntryCount = 1 first; 1 vs 1+2 tested separately	Volume per order on trader trades
7	Second-extreme tolerance and overshoot	0.10 test value, both denominators logged	More near-miss examples
8	Break-even, trailing, extended targets	Not implemented	Trader commentary or order modifications
9	Instrument and session filters	None; symbol and hour logged for later study	Larger sample per instrument
10	Impulse filter into the model	Not implemented; ARM bar shares logged	Losing arrival cases, labelled blind

15. MQL5 vs Python

Criterion	A · Pure MQL5 EA	B · MQL5 EA + Python engine in the live path
Reliability	One process, no bridge to fail	Bridge, second runtime, more failure modes
Latency	In-terminal ticks	Adds a hop; irrelevant on M5 but adds risk
Backtesting	Strategy Tester with real ticks runs the identical code	Python part cannot run in the Tester; logic would be duplicated
Deployment / broker compatibility	One .ex5 on any MT5 broker or VPS	Needs Python on the machine; the MetaTrader5 package is Windows-only
Alerting, drawing	Native	Still needs MQL5 for drawing
Logging, analytics, ML research	CSV/JSON out; weak analysis tools	Strong

> **Recommendation.** The whole live path — detection, drawing, alerts, orders — is one MQL5 EA built on a single detector library that the Strategy Tester also runs, so live and backtest cannot diverge. Python is used **offline only**: it reads the EA's CSV/JSON logs and screenshots for statistics, the human-vs-bot tables and later research. No LLM is in the decision path at any stage; a language model may be used to explain logged setups or analyse results after the fact.

Project structure

```
MQL5/Experts/TWER/TWER_EA.mq5                inputs, OnInit/OnTick/OnTimer/OnTradeTransaction, mode switch
MQL5/Include/TWER/Types.mqh                  enums and structs (section 12)
MQL5/Include/TWER/Params.mqh                 inputs -> parameter struct, parameter-set hash for log tagging
MQL5/Include/TWER/SymbolCfg.mqh              logical <-> broker names, digits, tick value, sessions
MQL5/Include/TWER/Swings.mqh                 causal swing detection, versions A-D
MQL5/Include/TWER/Detector.mqh               state machine S0-S14, one instance per (symbol, tf, direction)
MQL5/Include/TWER/Model.mqh  Rejection.mqh  BArm.mqh  Geometry.mqh  Quality.mqh
MQL5/Include/TWER/Draw.mqh                   object table of section 10, redraw from persisted state
MQL5/Include/TWER/Alerts.mqh                 transition -> draw -> log -> notify, rate limiter, Telegram
MQL5/Include/TWER/Logger.mqh                 setups.csv, transitions.csv, rejects.csv, screenshots
MQL5/Include/TWER/Persist.mqh                state files per setup, restart reconstruction
MQL5/Include/TWER/Risk.mqh  Exec.mqh         sizing, safety limits, idempotent order layer (V2-V3 only)
MQL5/Scripts/TWER/TWER_UnitTests.mq5         fixture bars -> expected transitions
MQL5/Files/TWER/{logs,shots,state,fixtures}/
research/ (Python, offline)                  load_logs.py, agreement.py, populations.py, report.ipynb
```

16. Events, backtest and look-ahead control

- **Bar analysis.** `OnTimer` (1 s) checks each enabled (symbol, timeframe) for a new bar time; only then is the detector advanced by exactly one closed bar. Missing bars are detected by time gaps; a gap inside an open setup marks it `DATA_GAP` and pauses it.
- **Tick monitoring.** `OnTick` / timer reads `SymbolInfoTick` only for setups in S9–S12: virtual or real fills, stop, target, missed. Nothing structural runs per tick.
- **Multi-symbol.** V1 runs one EA instance per chart so the drawing is on the chart being scanned. A scanner mode in the same EA iterates the symbol configuration from one chart and opens/draws on a chart when a setup reaches S5. Timeframes are independent populations; there is no higher-timeframe filter.
- **Symbol layer.** Logical names (EURUSD, GBPUSD, USDJPY, XAUUSD, XAGUSD, USTEC, US30, USOIL, BTCUSD) map to broker names with suffixes (for example `EURUSDm`). Digits, point, tick value, contract size, volume limits, stop level and sessions are read from `SymbolInfo*` at start and re-read daily.

Backtest

- The same EA runs in the MT5 Strategy Tester in **"Every tick based on real ticks"** mode. In ALERT_ONLY it keeps virtual orders and writes the same logs.
- **Look-ahead is prevented by construction:** the detector receives one closed bar at a time and has no access to later bars; swings, the frozen model, B and ARM all carry a `confirmTime` later than their own time; a unit test asserts that no logged value has a source time later than its confirm time; and a truncation test re-runs history cut at random points and requires identical output up to the cut.
- **Execution realism.** SELL limit fills when bid ≥ E; SELL stop triggers on ask ≥ SL; SELL target on ask ≤ B (mirror for BUY). Spread comes from the real ticks. Where only OHLC is available and one bar spans both stop and target, the stop is assumed first.
- **Output.** setups.csv with the fields of the research logger (wave points, model, rejection, B, ARM, entries, both stops, TP, R, grade, reasons, result, MAE, MFE, time to entry / B / stop, spread, slippage where real), plus transitions.csv and rejects.csv, each row tagged with the parameter-set hash.

17. Risk management, safety and recovery (V2–V3 only)

```
risk_money   = equity * RiskPercentPerSetup / 100
loss_per_lot = OrderCalcProfit(type, symbol, 1.0, E, SL)            # broker's own contract math, per entry
volume_k     = floor((risk_money / EntryCount) / |loss_per_lot| / volStep) * volStep
reject trade if volume_k < volMin, or > volMax, or margin check fails        -> reason STOP_INVALID / VOLUME
```

- **Limits.** Max open setups, max per symbol, max daily loss, max consecutive losses (then pause until manual reset), max spread at placement, max slippage on fill. A breached limit blocks new orders and raises an alert; it never closes positions by itself except the daily-loss stop if enabled.
- **Modes.** Master enable switch; ALERT_ONLY (default) → DEMO → LIVE. LIVE requires the input set to LIVE, a typed confirmation string, and the account not being flagged demo-only. `CLOSE ALL BOT POSITIONS` button closes positions and deletes pending orders carrying this EA's magic number only.
- **Idempotent orders.** Each order carries magic = base + hash(symbol, tf) and comment `TWER|setupId|E1`. Before any send: (1) the setup's persisted record must not already hold a ticket for that entry, (2) no live order or position with that magic and comment may exist, (3) the intent is written to disk *before* `OrderSend`, and the ticket after. One setup ID can place each entry once, regardless of how many ticks or restarts occur.
- **Errors.** Retcodes are classified: requote / off-quotes / connection → bounded retry while the limit price is still valid; invalid stops / volume / market closed → setup rejected with the code logged; no silent retries.
- **Restart recovery.** On init the EA loads state files, lists orders and positions by magic, matches them to setups by comment, rebuilds chart objects from state, and re-evaluates bars missed while offline one by one. A setup whose state cannot be reconciled is frozen as `RECOVERY_REVIEW` and alerts instead of trading.

18. Testing and validation plan

Level	What	Pass condition
Unit	Fixture bar series → expected transitions for: swing detection, pullback, second extreme, model, tail, body, rejection classes, B, ARM, entry, stop, TP, no-chase, each §8 reason code, grade, sizing, duplicate prevention	All fixtures pass; the win / loss / missed / six no-trade scenes already built for the training video are reused as the first fixtures
Causality	Truncation test and confirm-time assertion	Zero violations
Integration	Tester run on M5 real ticks, ALERT_ONLY; objects, logs and alert counts consistent	Every alert has objects; every setup has one terminal state
Ground truth A	Replay on the dates of the trader's documented trades (needs broker M1/tick history for Feb–Mar 2026)	Reported, not tuned to: how many of his trades the detector finds, and what else it flags
Ground truth B	Human-vs-bot on 30–50 setups you label blind, including at least 10 no-trade charts	See acceptance table
Demo forward	V2 virtual/demo orders, at least 4 weeks and 50 completed setups	No duplicate order, no unreconciled restart, log matches broker history

Human-vs-bot table and proposed acceptance thresholds

Columns: Setup ID · human detected? · bot detected? · same direction? · same MODEL? · same B? · same entry zone? · same stop? · same TP? · difference · reason.

Measure	Definition of agreement	Proposed threshold
Detection	Bot reaches S9 on a setup the human marked valid	≥ 80 %
Direction	Of matched setups	100 %
MODEL	Bar windows overlap ≥ 60 % and tail identical	≥ 80 %
B	Within 0.1 R	≥ 80 %
Entry zone	Each edge within max(1 spread, 0.05 R)	≥ 80 %
No-trade	Bot rejects a chart the human marked no-trade	≥ 80 %

The thresholds are project acceptance values (TEST VALUE). Mismatches are studied and explained before any parameter is touched; parameters are not tuned to raise agreement on the same sample that measures it.

Conditions required before automatic trading is enabled

1. This specification approved, including which test values are frozen for the first population.
2. All unit, causality and integration tests pass.
3. Human-vs-bot thresholds met on 30–50 blind setups, with every mismatch explained in writing.

4. Historical ALERT_ONLY backtest completed on real ticks for at least EURUSD M5, results reported per population without optimisation.
5. Demo forward test: ≥ 4 weeks and ≥ 50 completed setups, zero duplicate orders, zero unreconciled restarts, virtual results match broker demo history.
6. Stop-buffer question decided explicitly by you from the side-by-side logs, in the knowledge that it is unresolved.
7. Risk inputs set by you; LIVE mode activated manually; first live size at the broker minimum.

Roadmap

Phase	Deliverable	Exit criterion
A · Specification	This document	Your approval
B · Detector	Swings + state machine library, unit tests	Fixtures and causality tests pass
C · Visualisation	Object table of §10, dashboard	Chart tells the setup without the log
D · Alerts	13 alerts, push, optional Telegram, screenshots	Alert ↔ chart parity verified
E · Backtest	Tester runs, logs, Python report	Populations reported
F · Human vs bot	Agreement tables	Thresholds met, mismatches explained
G · Demo / paper	V2 virtual and demo orders	Demo criteria met
H · Risk engine	Sizing, limits, kill switch	Sizing unit tests pass on every symbol
I · Pending-order execution	V3 idempotent order layer	Restart and duplicate tests pass
J · Demo forward	V3 on demo	Stable for the agreed period
K · Optional small live	Minimum size	Only by your explicit activation

19. Answers to the twenty questions

1. Which parts can be fully automated today?
Swing and wave detection, the origin test, the H1/H2 comparison, model cluster extraction (tail, body edges), rejection sizing, B confirmation, ARM detection, limit-order geometry, target at B, no-chase cancellation, logging, drawing and alerts. All of these are computable once the test values in §13 are fixed.

2. Which parts remain discretionary or unresolved?
The stop buffer and the trader's apparent tolerance of tail breaches; the exact entry price rule; whether rejection is mandatory; minimum wave and pullback size; the model boundary; entry count and volume split; break-even, trailing and extended targets; instrument and session selection. They are parameters or are left out, never hidden defaults.

3. What exact rules must be frozen before coding?
Swing version A+B with RevMult and MinWave1ATR; pullback origin rule and depth range; SecondExtremeTolerance and its denominator; model band and candle limits; STRICT rejection thresholds; B confirmation rule; ArmStartFrac; entry method BODY_EDGE with EntryCount 1; the two stop variants to log; TP = B; the reason codes; the log schema.

4. What should the first alert-only prototype do?
Run on EURUSD M5, STRICT population, one instance per chart: detect, draw every element of §10 at the moment it becomes known, send the 13 alerts with setup IDs, keep virtual orders to record what would have happened, save screenshots and write the three logs. It places no orders.

5. What should the first automatic version NOT do?
No market orders, no chasing, no averaging beyond the configured entries, no trailing or break-even, no extended targets, no ARRIVAL-mode trades, no trading on symbols or timeframes that have not passed validation, no size above the configured risk, no decisions by a language model.

6. MQL5 only, or MQL5 + Python?
MQL5 only for everything that runs live or in the Tester. Python offline for analysing the logs. Reason: one code path for live and backtest, one file to deploy, nothing to break between processes.

7. How will you prevent look-ahead bias?
The detector is fed one closed bar at a time; every structural value carries a confirm time later than its own time; B and the model are published only on their confirming bars; execution uses ticks in chronological order; a truncation test proves that cutting history never changes earlier output.

8. How will you prevent duplicate orders?
Unique setup ID; one persisted ticket slot per entry; intent written before OrderSend; magic number and comment checked against live orders and positions before sending; alerts and orders are emitted only by state transitions, which cannot repeat.

9. How will you detect the MODEL in real time?
Once H2 qualifies, the cluster is the consecutive run of closed bars around the running extreme whose bodies reach the band below it. It is 'forming' while price stays there and becomes 'confirmed' and frozen on the first bar that closes below its lowest body, provided the candle count is within limits. A new high re-anchors it.

10. How will you determine B without knowing future candles?
B is the running low after the model, and it becomes B only when a later bar closes above it by the confirmation amount. Until then it is a candidate and nothing is drawn. If price makes a lower low before the ARM starts, B is revised and the revision is counted.

11. How will you detect ARM?
After B is confirmed, ARM starts on the first bar that closes at least ArmStartFrac of the way from B back to the first entry level. Its bar count and largest single-bar share are logged to separate a gradual return from a one-bar spike.

12. How will the bot determine the entry zone?
The zone is the model's body range: lowest body to highest body of the frozen cluster. Entry 1 is the near edge and Entry 2 the deeper edge. The percentage method computes alternative levels from B and the tail and is logged alongside for comparison.

13. How will the bot determine Stop Loss?
Beyond the model tail. Because the distance is unresolved, V1 records two stops for every setup — exact tail plus spread, and tail plus 0.2R — and reports both outcomes. Which one V3 uses is your decision after seeing those logs.

14. How will the bot calculate TP at B?
TP is the confirmed B price, shared by all entries of the setup, set as the order's take-profit. R:R is computed afterwards from entry, stop and B and is displayed, never used to move the target.

15. How will it notify me immediately?
On each state transition: terminal alert, MT5 mobile push with a compact one-line summary and setup ID, and optionally Telegram with the full fields and the chart screenshot. A rate limiter keeps within MetaQuotes limits.

16. How will it scan multiple currency pairs?
A symbol configuration layer maps logical to broker names and reads contract properties. One detector instance exists per (symbol, timeframe, direction). V1: one EA per chart. Scanner mode: one EA iterates all configured symbols on a timer and opens the chart when a setup reaches S5.

17. How will it distinguish a valid setup from a NO TRADE?
A setup is valid only if it passes every transition S1→S9 and the geometry checks. Any failure ends it with one reason code from §8, shown on the chart and written to rejects.csv.

18. How will it log every decision?
transitions.csv has one row per state change with times, prices and reason; setups.csv has one row per finished setup with all structure, levels, grade, result, MAE/MFE and timings; rejects.csv has every rejected candidate. Each row carries the parameter-set hash and population.

19. What validation sample is required before automatic trading?
30–50 blind human-labelled setups including at least 10 no-trade charts for the detector, then at least 4 weeks and 50 completed setups on demo. These establish that the bot reproduces the documented decisions and executes cleanly; they do not establish profitability, which would need a far larger sample.

20. What are the biggest risks of automating this strategy?
(1) The evidence base is 33 trades from one account and one month; the win-rate interval is roughly 35–68 %. (2) Several rules that drive the result — stop distance, entry price, rejection — are unresolved, so the bot tests our reconstruction, not necessarily the trader's method. (3) The trader appears to have used discretion on stops that a bot cannot copy. (4) Every ATR threshold here is ours; tuning them on small samples would fit noise. (5) Swing-confirmation lag can make the bot late relative to a human eye. (6) Small targets relative to spread on M1/M5, especially on indices, oil and crypto, and around news. (7) Synthetic CFD symbols differ by broker in contract size, sessions and stop levels.

> **Guiding order of work:** reproduce the strategy → make it deterministic → test it → validate it → then automate it. Validation first, visualisation second, automation later, optimisation last.